Tenant Onboarding — Operations Runbook

> **Audience:** Suparna Dey Platform operations team **Goal:** spin up a brand-new restaurant fork (a "tenant") from the `Curry-Co-Template` on GitHub, wire it into the Suparna Dey Platform (Super Admin), and get it into the App Store & Google Play. **Expected time:** ~30 min for the code + platform steps, several hours to a few days for store review.

---

0 · Prerequisites (one-time per operator)

Make sure you have logins for **all** of these before you start:

Account	Why	URL
GitHub	Fork the template repo	https://github.com
Emergent	Create the tenant's app project + hosting	https://emergent.sh
Suparna Dey Platform (Super Admin)	Register the tenant, issue API key, manage billing	(internal URL)
Stripe	The **hub** owns Stripe; you only need read-access to the platform account	https://dashboard.stripe.com
Google Cloud Console	Google Maps API key (unique per tenant)	https://console.cloud.google.com
Twilio	SMS OTP (optional; can defer)	https://console.twilio.com
Apple Developer	iOS build + App Store submission — **paid, $99/yr**	https://developer.apple.com
Google Play Console	Android build + Play Store submission — **paid, one-time $25**	https://play.google.com/console

Also gather from the **tenant** (use the *Tenant Intake Form* document — they fill it out, you file it here).

---

1 · Fork the template on GitHub

1. Sign in to GitHub.
2. Go to the **Curry-Co-Template** repo.
3. Click **Fork** (top right).
4. Owner: your GitHub org (e.g. `SuparnaDey`). Name: `restaurant-<slug>` (e.g. `restaurant-jomidari`).
5. **Un-tick** "Copy the `main` branch only" if you want to preserve history.
6. Click **Create fork**.

Copy the fork's HTTPS clone URL — you'll paste it into Emergent in the next step.

---

2 · Create the Emergent project + clone the fork

1. Sign in to Emergent (https://emergent.sh).
2. **New project** → give it a name matching the tenant (e.g. `Jomidari Restaurant`).
3. In the project chat, tell the agent:

> **Prompt to Emergent agent (project bootstrap):** ``` Import this GitHub repository as the starting point for the project: https://github.com//restaurant-
>
> Do NOT restructure or refactor anything. This is a working template. After clone, run the app and confirm it boots. Do not run tests yet. ```

1. Wait for the agent to finish. It should say "app is running" and give you a preview URL.

---

3 · Register the tenant in Super Admin

Before you touch any code, get the tenant credentials from the hub — you'll need them in the next step.

1. Open the Suparna Dey Platform Super Admin portal.
2. **Tenants** → **Add tenant**.
3. Fill in: - **Slug** — lowercase kebab-case, e.g. `jomidari` - **Legal / display name** — from the intake form - **Owner email** — the restaurant owner's email - **Region** — e.g. Canada / CAD / America/Toronto - **Default plan** — usually the platform default monthly plan

4. Click **Create**. The portal issues: - `tenant_id` — looks like `tnt_jomidari` - `api_key` — looks like `sk_tnt_...` (shown **once**; copy immediately)
5. Optional: give the tenant a **free trial** (7–30 days) so they can configure everything before their card is charged.

Store both values in your password manager under the tenant's record.

---

4 · Rebrand the tenant — one detailed prompt for the agent

Head back to the Emergent project chat for this tenant.

Step 1 — attach these files to the chat before you send the prompt: - `logo.png` - `logo-square.png` - `splash.png` (if the tenant provided one) - Any brand guideline PDF the tenant sent

Step 2 — paste this **exact prompt**, filling in the placeholders:

> **Prompt to Emergent agent (rebrand):**

```
Rebrand this template to a new tenant. Do NOT hand-edit UI strings —
only touch the two config files below and the assets folder.

── A · Drop the attached logos into place ──────────────────────────
Save the attachments into /app/frontend/assets/brand/ using the EXACT
filenames below (do NOT rename anything else in that folder):
  logo.png              — the horizontal / stacked logo
  logo-square.png       — the square mark (1024×1024 recommended)
  splash.png            — the splash screen (or reuse logo-square on a
                          solid background if the tenant didn't send one)
  notification-icon.png — a 96×96 white-on-transparent silhouette
                          (auto-generate from logo-square if not supplied)
  adaptive-icon.png     — 1024×1024, same content as logo-square

── B · Auto-derive the color palette from the logo ─────────────────
The tenant does NOT know hex codes. Extract the palette from
logo-square.png yourself using Pillow's Image.quantize / getcolors:

  from PIL import Image
  im = Image.open("/app/frontend/assets/brand/logo-square.png").convert("RGB")
  pal = im.quantize(colors=8).convert("RGB").getcolors(1_000_000)
  pal.sort(reverse=True)  # most-frequent first

Rules to pick the 4 brand colors:
  1. Drop any color whose luminance is > 0.92 (near-white — that's
     the logo background, not a brand color) or < 0.08 (near-black).
  2. colors.primary     = the most-frequent remaining color, converted
                          to hex.
  3. colors.primaryDark = the primary color darkened by ~20 %
                          (multiply RGB by 0.8, then to hex).
  4. colors.secondary   = the next most-frequent color that is NOT
                          within 30° hue of primary — this becomes
                          the accent gold/green/teal etc.
                          Fallback: shift primary by 180° hue.
  5. colors.accent      = a dark neutral for info states. Use
                          "#2F4F4F" (dark teal) as a safe default
                          unless the logo has a strong dark color
                          that meets contrast requirements.

Print the picked colors in your reply so I can eyeball them.

HOWEVER: if I supplied explicit color overrides below, use those and
skip the extraction:
  Primary override:   <blank | hex | plain-English from intake form>
  Accent override:    <blank | hex | plain-English from intake form>
  Colors to AVOID:    <blank | list>

── C · Edit /app/brand.json ────────────────────────────────────────
  key:                  "<slug, e.g. jomidari>"
  name:                 "<Display Name, e.g. Jomidari Kitchen>"
```

```
  legal_name:           "<Legal entity name>"
  email_from:           "orders@<tenant-domain>"
  admin_email:          "admin@<tenant-domain>"
  support_email:        "support@<tenant-domain>"
  guest_email_domain:   "guest.<tenant-domain>"
  delivery_admin_email: "delivery@<tenant-domain>"
  region.country:       "<CA|US|GB|IN>"
  region.currency:      "<CAD|USD|GBP|INR>"
  region.timezone:      "<IANA tz, e.g. America/Toronto>"
  platform.tenant_id:   "<tnt_... from Super Admin>"
  platform.api_key:     "<sk_tnt_... from Super Admin>"
  platform.status_url:  keep the existing value

── D · Edit /app/frontend/brand.config.js ──────────────────────
Same identity values as brand.json, PLUS:
  tagline:                "<one-line pitch>"
  colors.primary:         <hex you derived in step B>
  colors.primaryDark:     <hex you derived in step B>
  colors.secondary:       <hex you derived in step B>
  colors.accent:          <hex you derived in step B>
  support.email / phone / website / privacyUrl / termsUrl
  region.currencySymbol / defaultCity / serviceArea
  native.customer.bundleIdentifier: "com.<slug>.customer"
  native.customer.androidPackage:   "com.<slug>.customer"
  native.customer.deepLinkScheme:   "<slug>"
  native.delivery.bundleIdentifier: "com.<slug>.delivery"
  native.delivery.androidPackage:   "com.<slug>.delivery"
  native.delivery.deepLinkScheme:   "<slug>delivery"
  copy.aboutBlurb / deleteWarning / deleteFarewell / orderTagline

── E · Regenerate per-tenant secrets in /app/backend/.env ──────
  JWT_SECRET:              random 40+ char string
  INTEGRATIONS_FERNET_KEY:
    python -c "from cryptography.fernet import Fernet; \
               print(Fernet.generate_key().decode())"

── F · Do NOT touch these yet ──────────────────────────────────
Leave STRIPE_API_KEY, GOOGLE_MAPS_API_KEY, EMERGENT_EMAIL_KEY, TWILIO_*
blank — I'll paste them into the Emergent env panel after this rebrand.

── G · Deliverables ────────────────────────────────────────────
After you're done:
  1. Restart backend + Expo.
  2. Print the 4 derived hex codes so I can approve them.
  3. Screenshot the customer home screen.
  4. Screenshot the admin login screen.
```

If the derived palette looks off after the agent runs, tell it:

```
The primary color looks too <dark|light|orange|cool>. Please pick the
<Nth> most-frequent color from logo-square.png instead, or lighten /
darken the current primary by <X> %.
```

---

5 · Populate integration keys via Emergent's env panel

Not all keys need to come from your ops team. Below is exactly **which ones the platform auto-generates**, **which ones Emergent provides for you**, and **which ones you paste from an external provider account**.

5.1 · One-command secret generator

For every new tenant, run this script from the Emergent project shell. It creates every random secret at once, ready to copy-paste into the Emergent env panel:

```
python scripts/generate_tenant_secrets.py --slug <slug> --domain <tenant-domain>
```

Sample output:

```
── Auto-generated tenant secrets ──────────────────────────
JWT_SECRET              = ck8x_9Yq2WvT...
INTEGRATIONS_FERNET_KEY = <base64 fernet key>
ADMIN_EMAIL             = admin@<tenant-domain>
ADMIN_PASSWORD          = 3f8Q!kL9pW#zR7mN
REVIEWER_EMAIL          = reviewer@<tenant-domain>
REVIEWER_PASSWORD       = 8m2N!bV6xQ#tK4jP
EMAIL_FROM              = orders@<tenant-domain>
EMAIL_FROM_NAME         = <brand.name from brand.json>
DB_NAME                 = <slug>_db
```

Copy the whole block into the Emergent env panel. That's 8 fields done in one shot — **your ops team never has to invent a password or think about entropy**.

5.2 · Emergent-managed keys (already available, just enable)

These three keys live inside your Emergent account and are shared across all your projects — you never rotate them per tenant. Just tell the Emergent agent:

Prompt to Emergent agent (attach integrations):

```
Attach the Emergent-managed integrations to this project:
  1. EMERGENT_LLM_KEY   — universal LLM key from my profile
  2. EMERGENT_EMAIL_KEY — Emergent-managed Resend integration
  3. EMERGENT_PUSH_KEY  — leave as "placeholder", it auto-fills on publish

Set them in /app/backend/.env and confirm the backend starts cleanly.
```

The agent handles it in one step — no dashboard clicks required.

5.3 · Google Maps API key — why we need it, how to get it

Why we need it:

Feature	Google Maps API used
Customer types their delivery address → autocomplete dropdown appears	**Places API (Autocomplete)**
We convert the picked address to lat/lng for the driver	**Geocoding API**
Customer sees the live driver dot moving on a map during delivery	**Maps JavaScript API** (web) + **Maps SDK for Android/iOS** (native)
Driver's route from kitchen → customer with ETA	**Directions API**
Delivery-fee zoning by distance	**Distance Matrix API**

If any of these are missing the app still works, but the address flow degrades to a manual text-only form — a real UX regression.

Can we use ONE key across all tenants? Yes — and we recommend it.

The cleanest architecture is **one Google Cloud project owned by Suparna Dey**, with **one API key restricted to every tenant's bundle IDs + web domain**. Advantages:

- One bill, one quota dashboard, one place to spot abuse
- You can revoke a single tenant's access without touching others (remove their bundle ID from the key's allow-list)
- No need to hand a Google Cloud login to the tenant

One-time Google Cloud setup (do this once, ever):

1. https://console.cloud.google.com → **New project** → name it `Suparna Dey Platform` → **Create**.

2. **Billing** → attach a payment method. Google gives $200/month free Maps credit — most tenants sit inside that.
3. **APIs & Services** → **Library** → enable each of these: - Maps JavaScript API - Maps SDK for Android - Maps SDK for iOS - Places API - Geocoding API - Directions API - Distance Matrix API
4. **APIs & Services** → **Credentials** → **+ Create credentials** → **API key**. Copy it — this is your **platform-wide** `GOOGLE_MAPS_API_KEY`.
5. Click the key → **Application restrictions** → **Websites**: leave open for now (you'll add domains as tenants launch). **API restrictions**: restrict to the 7 APIs above.

Per-tenant onboarding (each new tenant):

Go back to the key from step 4 and edit the allow-lists:

- **Under HTTP referrers**, add:
- `<slug>.emergent.host/*`
- `<slug>-preview.emergent.host/*` (preview URL, before custom domain)
- Any custom domain the tenant owns, e.g. `*.jomidari.com/*`
- **Under Android apps**, add:
- Package name: `com.<slug>.customer` + your app's SHA-1 fingerprint (Emergent shows this on the Android build page)
- Package name: `com.<slug>.delivery` + SHA-1
- **Under iOS apps**, add:
- Bundle ID: `com.<slug>.customer`
- Bundle ID: `com.<slug>.delivery`

Then paste **the same** `GOOGLE_MAPS_API_KEY` into the tenant's Emergent env panel. That's it — you never generate a new key.

> **Cost heads-up**: at $200/mo free credit you comfortably support ~28,000 map loads and ~2,800 autocomplete sessions per month, per project. If any single tenant spikes above that, split them into their own key later — the switch is a 1-line env edit + rebuild.

5.4 · Twilio (SMS OTP) — optional, one shared account

Same pattern as Google Maps: one Twilio account, one **Messaging Service SID** per tenant (so texts show the tenant's brand name, not "Suparna Dey").

- **One-time**: sign up at twilio.com, buy one phone number per country you operate in, note the `TWILIO_ACCOUNT_SID` and `TWILIO_AUTH_TOKEN`.

- **Per tenant**: in the Twilio console → **Messaging** → **Services** → **Create** — name it `<slug>-sms`. Assign the country phone number to it. Copy the **Messaging Service SID** (`MG...`).
- In the tenant's Emergent env panel set:
- `TWILIO_ACCOUNT_SID` — shared
- `TWILIO_AUTH_TOKEN` — shared
- `TWILIO_MESSAGING_SERVICE_SID` — **per tenant**

Skip this entirely if the tenant doesn't want SMS at launch — the app falls back to email OTPs.

5.5 · Stripe — leave blank on the tenant

The **hub** owns Stripe. Tenants never set `STRIPE_API_KEY`. Leave it empty in the env panel.

5.6 · Final env panel checklist (per tenant)

Copy this table into your ticketing system as the "keys done?" checklist for every onboarding:

Var	Where it comes from	Who provides it
`MONGO_URL`	Auto-populated by Emergent	Emergent
`DB_NAME`	Output of `generate_tenant_secrets.py`	Auto
`JWT_SECRET`	Output of `generate_tenant_secrets.py`	Auto
`INTEGRATIONS_FERNET_KEY`	Output of `generate_tenant_secrets.py`	Auto
`ADMIN_EMAIL`	Output of `generate_tenant_secrets.py`	Auto
`ADMIN_PASSWORD`	Output of `generate_tenant_secrets.py`	Auto
`REVIEWER_EMAIL`	Output of `generate_tenant_secrets.py`	Auto
`REVIEWER_PASSWORD`	Output of `generate_tenant_secrets.py`	Auto
`EMAIL_FROM` / `EMAIL_FROM_NAME`	Output of `generate_tenant_secrets.py`	Auto
`EMERGENT_LLM_KEY`	Emergent agent attaches	Emergent
`EMERGENT_EMAIL_KEY`	Emergent agent attaches	Emergent
`EMERGENT_PUSH_KEY`	Placeholder, auto-fills on publish	Emergent
`STRIPE_API_KEY`	LEAVE BLANK — the hub owns Stripe	—
`GOOGLE_MAPS_API_KEY`	Paste the ONE platform-wide key from §5.3	Ops
`TWILIO_ACCOUNT_SID`	Paste the shared account SID from §5.4	Ops
`TWILIO_AUTH_TOKEN`	Paste the shared token from §5.4	Ops
`TWILIO_MESSAGING_SERVICE_SID`	Create per-tenant service in §5.4	Ops
`PUBLIC_BASE_URL`	Empty for now; fill after first Emergent deploy	Ops (deploy step)

In total your ops team pastes at most 4 values (Google Maps + 3 Twilio) — everything else is auto-generated or Emergent-provisioned.

Restart the backend after saving (Emergent panel button). Give the `ADMIN_PASSWORD` to the tenant owner via 1Password/Bitwarden — never over email.

---

6 · Seed the menu (fastest path: import from another tenant)

If the tenant is starting from scratch, use the seed pack from Curry & Co. as a template.

1. From the Emergent project's file browser, open `/app/docs/seed-packs/curryco_seed_pack.json`.
2. Duplicate it as `/app/docs/seed-packs/<slug>_seed_pack.json`.
3. Hand-edit the item names, prices, and images to the tenant's menu.
4. In the Emergent chat, tell the agent:

> `Import /app/docs/seed-packs/<slug>_seed_pack.json into MongoDB using /app/backend/tools/seed_import.py. Overwrite any existing menu. Then hit POST /api/dev/seed once to publish the current week.`

For a truly empty start, skip the import and use the Admin portal's **Master Menu** → **Add item** flow after the first deploy.

---

7 · Smoke test in preview

Have the agent open the Expo preview URL. Manually verify:

1. **Customer app** — home screen shows tenant logo + tagline + brand colors.
2. **Admin portal** at `/admin` — log in with `ADMIN_EMAIL` / `ADMIN_PASSWORD`.
3. **Delivery portal** at `/delivery-partner` — seed a driver (Admin → Delivery → Invite).
4. **Billing tab** at `/admin/billing` — should show **"Billing is coming soon"** or a real plan card if the hub has already provisioned this tenant.
5. **Platform banner** — if the tenant has an unpaid subscription in the hub, the red banner should appear at the top of the Admin portal.

If any of the above breaks, tell the agent:

> `The <thing> is broken on the customer/admin/delivery app. Please investigate the last 100 backend logs and fix, then re-verify.`

---

8 · First deploy (production preview)

1. In the Emergent project, click **Publish** (top-right).
2. Emergent asks for a subdomain — use `<slug>.emergent.host` or a custom domain the tenant owns.
3. Wait for the build to finish (~2–5 min).
4. Once deployed, note the public URL. Set `PUBLIC_BASE_URL` in the env panel to this URL and click **Redeploy**.
5. From your phone, open the URL and verify: - Logo + colors render correctly - Google Maps map tiles load (address autocomplete works in checkout) - Placing a test order goes through (or is blocked with the right message if the tenant's SaaS subscription isn't active yet).

---

9 · Wire billing (tenant Admin subscribes to Suparna Dey plan)

Only needed if the tenant is starting paid immediately (skip during trial).

1. The tenant owner logs into their Admin portal → **Billing** tab.
2. Clicks **Subscribe** → redirected to Stripe Checkout hosted by the hub.
3. Enters card → returns to the Billing tab; status flips to **ACTIVE**.
4. The 5-min phone-home poller picks up the new status; the "Fix billing" banner (if it was showing) disappears within 5 min.

You can watch the transition live in the Super Admin portal → Tenants → Jomidari → Subscription tab.

---

10 · iOS build & App Store submission

10.1 · One-time Apple Developer setup for this tenant

1. In **App Store Connect** → **My Apps** → **New App**.
2. **Bundle ID** — must match `native.customer.bundleIdentifier` from brand.config.js exactly, e.g. `com.jomidari.customer`. Create it in **Certificates, IDs & Profiles** → **Identifiers** → **+**.
3. **Name** — the tenant's public brand name.
4. **Primary language** — English (or per intake form).
5. **SKU** — `<slug>-customer-01`.
6. Repeat for the **delivery partner** app: bundle ID `com.jomidari.delivery`, SKU `<slug>-delivery-01`.

10.2 · Generate the iOS build in Emergent

1. In the Emergent project, click **Publish → Generate iOS build**.
2. Emergent asks for: - Apple Team ID (from your Apple Developer account membership page) - Apple ID email + app-specific password
3. Emergent handles the signing + build (~10–20 min).
4. When done, Emergent submits the `.ipa` to TestFlight.

10.3 · TestFlight → App Store

1. Wait for Apple's automated processing (5–30 min).
2. In App Store Connect → TestFlight, add the tenant owner as an internal tester.
3. Owner installs TestFlight on their phone, tests the app.
4. When approved by the owner, add a **new version** in App Store Connect → complete **App Information**, **Pricing**, **App Privacy**, **App Review Information** (use the reviewer creds from the intake form), attach 5–10 **screenshots** (6.7", 6.5", 5.5" required for iOS).
5. Submit for review. Apple takes 24–72 hours; expect at least one rejection cycle on the first submission.

10.4 · Common iOS rejection reasons and fixes

Reason	Fix
4.0 Design (feels like a website)	Add native iOS gestures (already in template — verify safe-area)
5.1.1 Privacy (missing usage descriptions)	Check `expo.ios.infoPlist` in `/app/frontend/app.json`
4.3 Spam (looks like other apps)	Emphasise the tenant's unique brand & menu in the app description
2.1 App Completeness (login failure)	Give the reviewer working credentials in the review notes

---

11 · Android build & Play Store submission

11.1 · One-time Play Console setup

1. **Google Play Console → Create app**.

2. **App name** — tenant's brand name.
3. **Package name** — must match `native.customer.androidPackage`, e.g. `com.jomidari.customer`.
4. **App or game** → App. **Free or paid** → Free.
5. Repeat for the **delivery partner** app.

11.2 · Generate the Android build in Emergent

1. Emergent → **Publish** → **Generate Android build**.
2. Emergent produces both `.APK` (for sideload testing) and `.AAB` (for Play Store upload).
3. Download both from the Emergent build panel.

11.3 · Sideload test on a real device

1. Enable **Developer mode** → **USB debugging** on an Android phone.
2. `adb install <path-to>.apk` (or open the APK from the phone's file manager).
3. Smoke-test the customer + delivery apps.

11.4 · Upload to Play Store

1. Play Console → **Production** → **Create new release**.
2. Upload the `.AAB` file.
3. Fill in: - **Release notes** - **Store listing** — description, short description, feature graphic (1024×500), phone screenshots (min 2) - **Privacy policy URL** (from brand.json → support.privacyUrl) - **Data safety** form - **Content rating** — take the questionnaire (usually PEGI 3 / IARC E)
4. Roll out to **internal testing** first (add tenant owner as tester).
5. When approved, promote to **production**.

11.5 · Common Play rejection reasons

Reason	Fix
Data safety mismatch	Update the form: we collect email + name + address + order history
Missing privacy policy	Set `support.privacyUrl` in brand.config.js, redeploy, resubmit
Screenshots don't match the app	Regenerate from the deployed app on a real device

---

12 · Ongoing operations

Daily / weekly

- Check the **Super Admin dashboard** for any tenants with `past_due` or `unpaid` status — reach out proactively before the block kicks in.
- Watch the Emergent project's logs for backend errors (Errors tab).

When the tenant asks for a change

- Menu / prices / catering → they do it themselves in the Admin portal.
- Colors / logos / tagline → you (ops) edit `brand.json` and `brand.config.js`, then redeploy.
- Adding a new integration → go through the standard Emergent agent chat + Integrations Playbook flow.

When rolling out platform-wide fixes

- Fixes land in the `Curry-Co-Template` repo → merge into each tenant's fork (`git fetch upstream && git merge upstream/main`).
- Restart the tenant's backend + expo services from the Emergent panel.

When a tenant offboards

- Cancel their subscription in Super Admin → their app auto-suspends within 5 min via the phone-home poller.
- Remove them from the App Store / Play Store (transfer to the tenant's own developer account first if they want to keep the app running independently).
- Archive the GitHub fork.

13 · Emergency runbook

Symptom	First thing to check
Tenant says "customers can't order"	Super Admin → subscription status. If suspended, they missed a payment.
Tenant says "delivery drivers can't log in"	Same as above — the driver app is gated on subscription too.
Tenant says "the app looks like Curry & Co."	`brand.json` / `brand.config.js` values weren't updated. Re-run Step 4.
Tenant says "orders aren't reaching the kitchen"	Check the Admin portal's Orders tab; check backend logs for 402/500s.
Tenant says "notifications don't work"	Verify build is production (Expo Go can't receive push). Check `google-services.json` / APN keys.
Tenant says "Stripe checkout is broken"	The **tenant** never had Stripe. This is the **hub's** Stripe. Confirm hub is up.

14 · Quick reference — files that always need editing

The **only** files you should ever hand-edit per tenant:

```
/app/brand.json                          ← identity, contact, region, platform creds
/app/frontend/brand.config.js            ← same values + design tokens
/app/frontend/assets/brand/*.png         ← logos & splash
/app/backend/.env                        ← integration keys (via Emergent panel, not git)
/app/docs/seed-packs/<slug>.json         ← initial menu (optional)
```

Helper scripts you'll re-use every onboarding:

```
scripts/generate_tenant_secrets.py    ← auto-generate all random secrets (§5.1)
scripts/scrub_for_template.py         ← wipe tenant creds before pushing to Template repo
scripts/build_docs.py                 ← rebuild these ops PDFs after editing the MDs
```

Everything else in the codebase is tenant-agnostic. If you find yourself editing anything outside this list, stop and flag it in the `#platform-eng` channel — it likely belongs in one of the config files above and is a bug in the template.

---

15 · Handoff checklist

Before you consider a tenant "onboarded", confirm:

- [ ] GitHub fork exists under our org
- [ ] Emergent project created & bootstrapped
- [ ] Super Admin tenant record created; `tenant_id` + `api_key` stored
- [ ] `brand.json` + `brand.config.js` populated
- [ ] Logo assets in `/app/frontend/assets/brand/` swapped
- [ ] Integration keys set in Emergent env panel
- [ ] Menu seeded (or blank + admin-owner trained on how to add)
- [ ] Preview URL smoke-tested end-to-end
- [ ] Production URL deployed
- [ ] Billing tab tested (or scheduled during trial)
- [ ] iOS bundle IDs created + first build submitted
- [ ] Android package names created + first `.AAB` uploaded
- [ ] Reviewer creds handed to Apple + Google
- [ ] Owner has Admin creds in their password manager
- [ ] Owner walked through the Admin portal in a live 30-min call
- [ ] Support contact + escalation path shared with the owner

Once every box is ticked, mark the tenant **LIVE** in the Super Admin portal and move them from onboarding to steady-state ops.